

How to assign from a function which returns more than one value?

Asked 14 years, 9 months ago · Modified 11 months ago · Viewed 331k times ·  Part of R Language Collective

Still trying to get into the R logic... what is the "best" way to unpack (an LHS) the results from a function returning multiple values?

I can't do this apparently:

```
R> functionReturningTwoValues <- function() { return(c(1, 2)) }
R> functionReturningTwoValues()
[1] 1 2
R> a, b <- functionReturningTwoValues()
Error: unexpected ',' in "a,"
R> c(a, b) <- functionReturningTwoValues()
Error in c(a, b) <- functionReturningTwoValues() : object 'a' not found
```

must I really do the following?

```
R> r <- functionReturningTwoValues()
R> a <- r[1]; b <- r[2]
```

or would the R programmer write something more like this:

```
R> functionReturningTwoValues <- function() { return(list(first=1, second=2)) }
R> r <- functionReturningTwoValues()
R> r$first
[1] 1
R> r$second
[1] 2
```

--- edited to answer Shane's questions ---

I don't really need giving names to the result value parts. I am applying one aggregate function to the first component and an other to the second component (`sum` and `max` , if it was the same function for both components I would not need splitting them).

    Edit tags

Share Edit Follow Close Flag

edited Oct 30, 2016 at 8:33 asked Dec 1, 2009 at 14:27
 33.7k 21 116 149  5,648 9 49 70

7  191, another way to return multiple values is to set an `attr` on your return value. – Jonathan Chang Dec 1, 2009 at 15:49

2  16, This is the equivalent of Python's tuple-unpacking. – smci Mar 20, 2015 at 11:58

18 Answers

Sorted by
 Data modified (newest first)

 The `keas` library has similar functionality to `zeallot`

```
library(keas)
functionReturningTwoValues <- function() { return(c(1, 2)) }
c(a, b) %>% functionReturningTwoValues()
```

   

answered Oct 6, 2023 at 4:19
 11 3

 This is only for the sake of completeness and not because I personally prefer it. You can pipe `%>%` the result, evaluate it with curly braces `{}` and write variables to the parent environment using double-arrow `<=>`.

```
library(tidyverse)
functionReturningTwoValues() %>% {a <=> {1}; b <=> {2}}
```

 **UPDATE:** You can also use the multiple assignment operator from the `zeallot` package: `%>%`

```
c(a, b) %>% List(0, 1)
```

   

edited Jun 2, 2022 at 9:57 answered May 26, 2022 at 16:44
 2,372 5 23 49

 Year 2021 and this is something I frequently use.

 `tidyverse` package has a function called `lst` that assigns name to the list elements when creating the list. Post which I use `listenv()` to assign variable or use the list directly

```
library(tidyverse)
env <- function() {
  a <- 1
  b <- 2
  list(a, b)
}
listenv(env(), envir = GlobalEnv) #unpacks list key-values to variable-values into the current environment
```

   

edited Nov 2, 2021 at 7:06 answered Nov 2, 2021 at 6:24
 23.9k 41 93 115  111 21 1

 I will post a function that returns multiple objects by way of vectors:

```
Median <- function(x){
  x.sort <- sort(x)
  if (length(x)%2==0){
    Median <- (x.sort[length(x)/2]+x.sort[length(x)/2+1])/2
  } else{
    Median <- x.sort[(length(x)+1)/2]
  }
  return(Median)
}
```

That was a function I created to calculate the median. I know that there's an inbuilt function in R called `median()` but nonetheless I programmed it to build other function to calculate the quartiles of a numeric data-set by using the `median()` function I just programmed. The `Median()` function works like this:

1. If a numeric vector `x` has an even number of elements (i.e., `length(x)%2==0`), the median is calculated by averaging the elements `x[n/2]` [`length(x)/2`] and `x[(n/2)+1]` [`sort(x)[length(x)/2+1]`].
2. If `x` doesn't have an even number of elements, the median is `sort(x)[length(x)+1/2]`.

On to the `QuartilesFunction()`:

```
QuartilesFunction <- function(x){
  x.sort <- sort(x) # Data is sorted in ascending order
  if (length(x)%2==0){
    # Data number is even
    halFNB <- x.sort[(length(x)/2)]
    halFUP <- x.sort[(length(x)/2+1):length(x)]
    Q1 <- Median(halFNB)
    Q3 <- Median(halFUP)
    Q1 <- Q1
    Q3 <- Q3
    Q2 <- Q1
    Q4 <- Q3
    Q2 <- Q2
    Q4 <- Q4
    Q2 <- Q1
    Q4 <- Q3
    Quartiles <- c(Q1, Q2, Q3, Q4)
    names(Quartiles) = c("Q1", "Q2", "Q3", "Q4")
  } else{ # Data number is odd
    # Including the median
    halFNB <- x.sort[(length(x)+1)/2]
    halFUP <- x.sort[(length(x)+1/2):length(x)]
    Q1 <- Median(halFNB)
    Q3 <- Median(halFUP)
    Q1 <- Q1
    Q3 <- Q3
    # Not including the median
    halFNB <- x.sort[(length(x)+2)/2-1]
    halFUP <- x.sort[(length(x)+1/2+1):length(x)]
    Q1 <- Median(halFNB)
    Q3 <- Median(halFUP)
    Q2 <- Median(halFNB)
    Q4 <- Median(halFUP)
    # Methods (1) and (2) averaged
    Q1 <- (Q1+Q2)/2
    Q3 <- (Q3+Q4)/2
    Q2 <- (Q2+Q3)/2
    Quartiles <- c(Q1, Q2, Q3, Q4)
    names(Quartiles) = c("Q1", "Q2", "Q3", "Q4")
  }
  return(Quartiles)
}
```

This function returns the quartiles of a numeric vector by using three methods:

1. Discarding the median for the calculation of the quartiles when the number of elements of the numeric vector `x` is odd.
2. Keeping the median for the calculation of the quartiles when the number of elements of the numeric vector `x` is odd.
3. Averaging the results obtained by using methods 1 and 2.

When the number of elements in the numeric vector `x` is even, the three methods coincide.

The result of the `QuartilesFunction()` is a vector that depicts the first and third quartiles calculated by using the three methods outlined.

   

answered Sep 1, 2021 at 19:06
 358 3 10

 With R 3.6.1, I can do the following

```
r2v <- function() { c(5,3) }
a,b <- r2v()
c(a,b[[1]]) # prints "5"
c(a,b[[2]]) # prints "3"
```

    

answered Oct 20, 2019 at 17:40
 4,161 2 33 47

 **(1)** `list[...]` <= > I had posted this over a decade ago on [r-help](#). Since then it has been added to the `grubfr` package. It does not require a special operator but does require that the left hand side be written using `list[...]` like this:

```
library(grubfr) # need 0.7-0 or later
list(a, b) <- functionReturningTwoValues()
```

 If you only need the first or second component these all work too:

```
list(a) <- functionReturningTwoValues()
list(a, ) <- functionReturningTwoValues()
list(), b <- functionReturningTwoValues()
```

(Of course, if you only needed one value then `functionReturningTwoValues()[1]` or `functionReturningTwoValues()[2]` would be sufficient)

See the cited r-help thread for more examples.

(2) **with** If the intent is merely to combine the multiple values subsequently and the return values are named then a simple alternative is to use `with` :

```
myfun <- function() list(a = 1, b = 2)

list(a, b) <- myfun()
a + b

# same
with(myfun(), a + b)
```

(3) **attach** Another alternative is `attach`:

```
attach(myfun())
a + b
```

ADDED: `with` and `attach`.

   

edited May 10, 2018 at 1:04 answered Feb 28, 2013 at 16:23
 6,568 18 216 356

30  1, I accepted your answer because of the "with", but I can't reproduce what you describe for the left hand side usage of "list", all I get is "object 'x' not found"

 **merlino** Mar 23, 2013 at 15:42

4  16, It works for me. What did you try? Did you read the linked post and follow it? Did you define `list` and `[<=>]` as shown there? – G. Grothendieck Mar 23, 2013 at 16:00

13  16, @G.Grothendieck, Would you mind if I put the content of your link into your answer? I think it would make it easier for people to use it. – merlino Apr 14, 2014 at 7:29

15  1, I agree with @merlino2011, as written it seems like this syntax is embedded into R base. – knowah Jun 3, 2014 at 22:52

7  16, @G.Grothendieck I agree with merlino2011 and knowah - it would be best if the actual code that is important here (the code referenced in the link) is in the answer. R might not be a bad idea to mention that the result object doesn't need to be named list. That confused me for a little while before reading your actual code. As mentioned the answer says that you need to run the code in the link but most people aren't going to read that code right away unless it is in the answer directly - this gives the impression that this syntax is in base R. – Dixon Jul 22, 2014 at 17:08



 I put together an R package `zeallot` to tackle this problem. `zeallot` includes a multiple assignment or unpacking assignment operator, `%>%`. The LHS of the operator is any number of variables to assign, built using calls to `c()`. The RHS of the operator is a vector, list, data frame, data object, or any custom object with an implemented `destructure` method (see `?zeallot::destructure`).

Here are a handful of examples based on the original post,

    

```
library(zeallot)
functionReturningTwoValues <- function() {
  return(c(1, 2))
}

c(a, b) %>% functionReturningTwoValues()
# # 1
# # 2

functionReturningListValues <- function() {
  return(list(1, 2))
}

c(d, e, f) %>% functionReturningListValues()
# # 1
# # 2
# # 3

functionReturningNestedList <- function() {
  return(list(1, list(2, 3)))
}

c(f, e(g, h)) %>% functionReturningNestedList()
# # 1
# # 2
# # 3

functionReturningToManyValues <- function() {
  return(as.list(1:20))
}
```


Share

Edit

Follow

Flag

answered Dec 1, 2009 at 15:01

Federico Giorgi

21110.7k94356

2

46

What if instead of `output <- func(5)` I want to have the result in two objects? I've tried with `list("a","b") <- func(5)` but it doesn't work. – skan Jul 5, 2017 at 13:49

Pin

Yes to your second and third questions -- that's what you need to do as you cannot have multiple 'lvalues' on the left of an assignment.

4

Share

Edit

Follow

Flag

answered Dec 1, 2009 at 14:41

Dik is no longer here

347k58657736

This post is hidden. It was deleted 11 years ago by the post author.

Another solution, in the favor of [the := operator referenced in a different answer](#), is to have your function return a list with names, like this:

```
xx <- function() {
  print("Called.")
  return(list=a, b=2)
}
```

You can then write a wrapper function that iterates through the returned list and uses `parent.frame` and `assign` to create the variables. Something like:

```
return.local <- function( list.to.upgrade ) {
  lapply( names(list.to.upgrade), function(which.name) {
    assign( which.name, list.to.upgrade[[which.name]], envir=parent.frame(3) )
  })
  return(MUL)
}
```

You'd use it like this: `return.local(xx())` - A full example:

```
> test()
[1] "return.local" "xx"
> return.local(test())
[1] "Called."
MUL
> test()
[1] "a"          "b"          "return.local" "xx"
> print(paste(a,b))
[1] "1 2"
```

This is a little non-intuitive because the names of the variables are declared "inside" the function. It works well for me though.

Share

Edit

Follow

Undo

Flag

edited May 23, 2017 at 11:47

Community

answered Apr 24, 2013 at 2:46

Nathan VanHousden

1113.3k22429

Comments disabled on deleted / locked posts / reviews

https://stackoverflow.com/questions/1026519/how-to-assign-from-a-function-which-returns-more-than-one-value

3/3